
LAPORAN PRAKTIKUM

INTERNET OF THINGS II

TUGAS 1



Nama :

Nama : Fadillah Akbar

NPM : 25782039

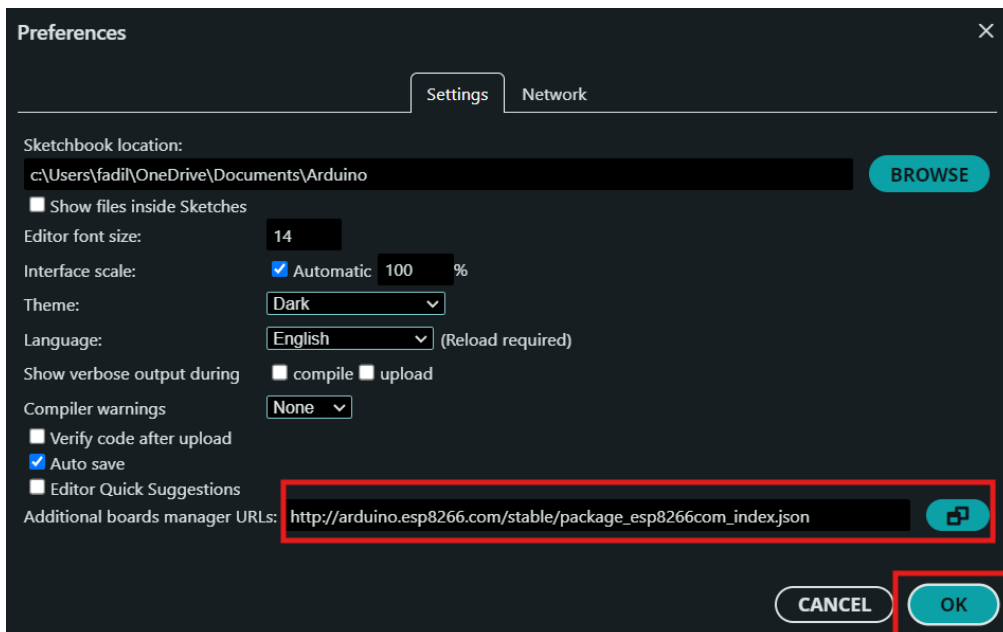
Kelas : TRI 3B

Dosen Pengampu : Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.
Agus Ambarwari, S.Pd., M.Kom.
Andi Saptono, A.Md

JURUSAN TEKNOLOGI INFORMASI
TEKNOLOGI REKAYASA INTERNET
POLITEKNIK NEGERI LAMPUNG
TAHUN 2026

INSTALASI Arduino.IDE

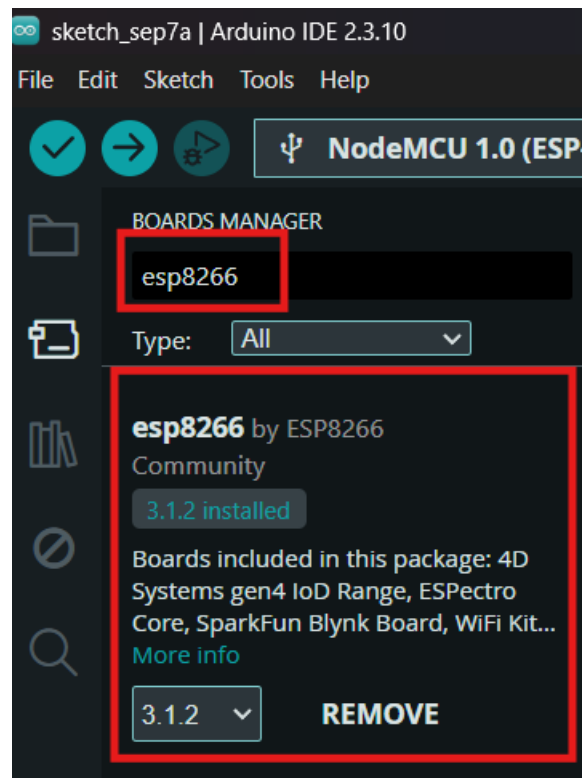
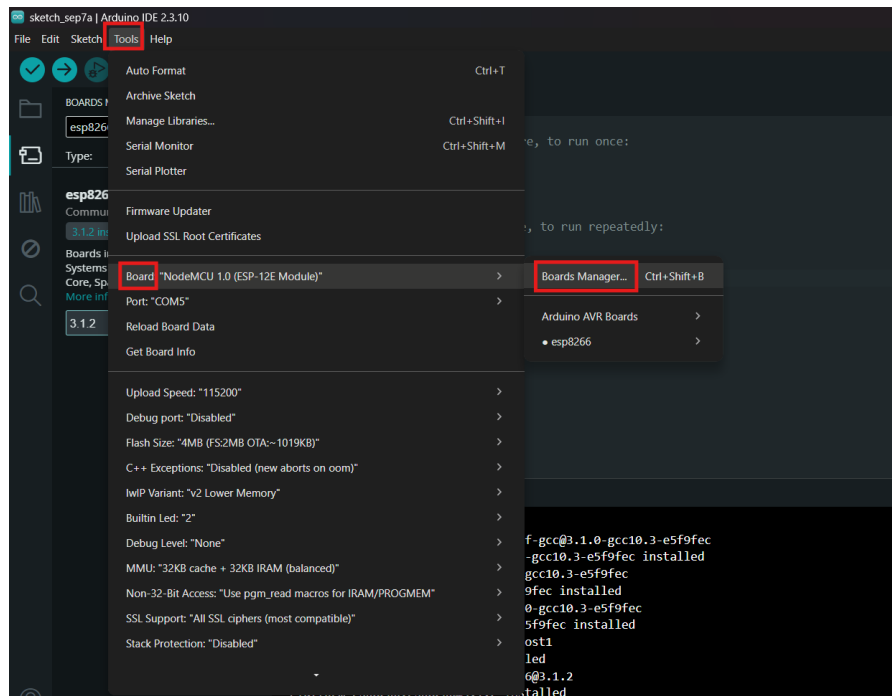
Sebelum memprogram, instal *board package* ESP8266 pada perangkat lunak Arduino Ide agar dapat dikenali.



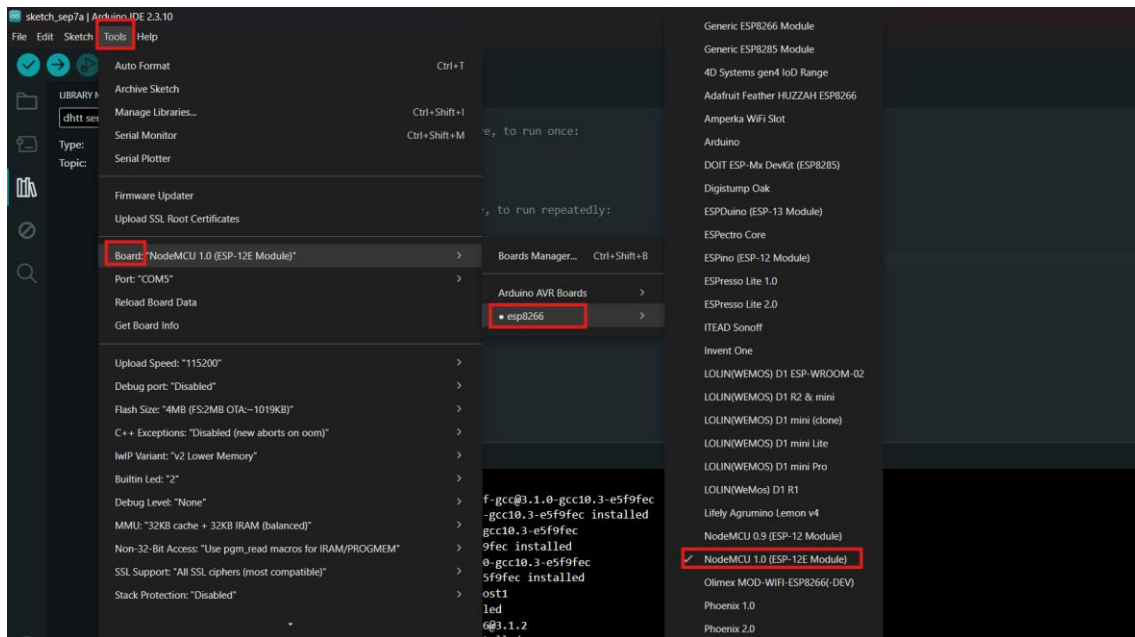
File > Preferences >

Lalu Tempel: http://arduino.esp8266.com/stable/package_esp8266com_index.json

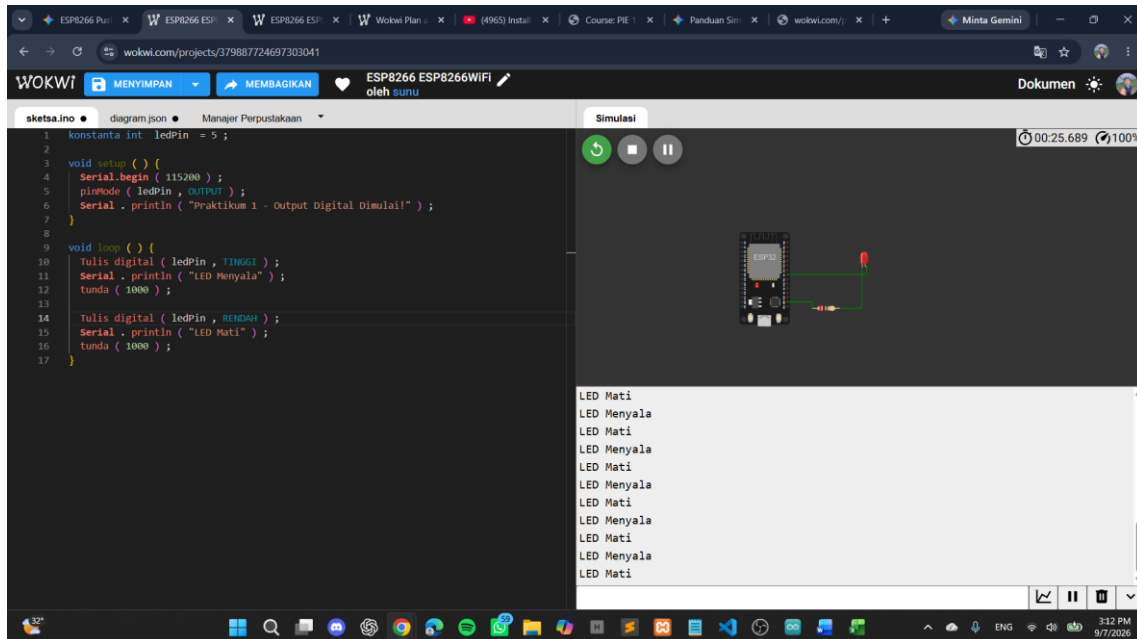
Lalu Install board ESP8266



Setelah board terinstall ikuti langkah langkah di bawah ini dan sesuaikan port dengan perangkat esp8266 nya

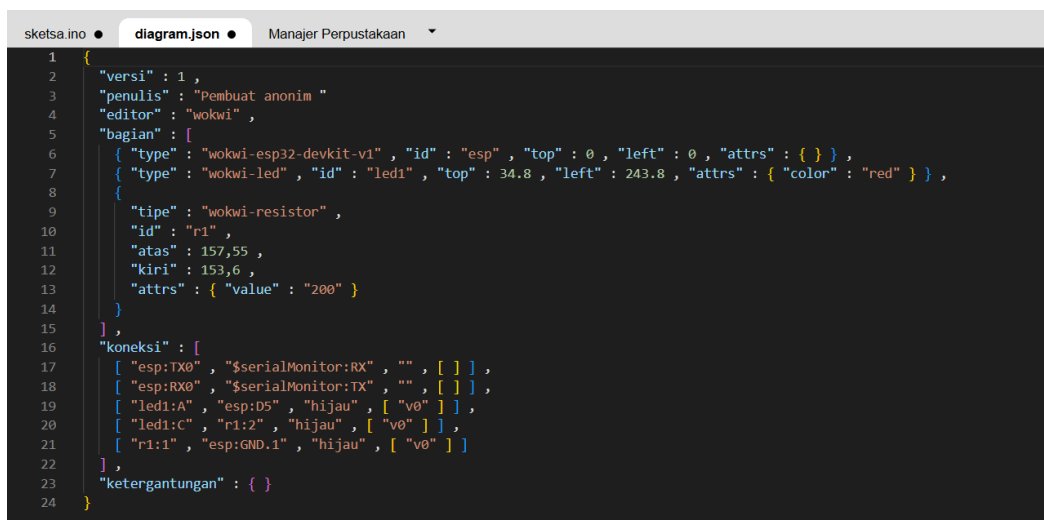


1.5.1 Praktikum 1: *Digital Output* (Menyalakan LED)

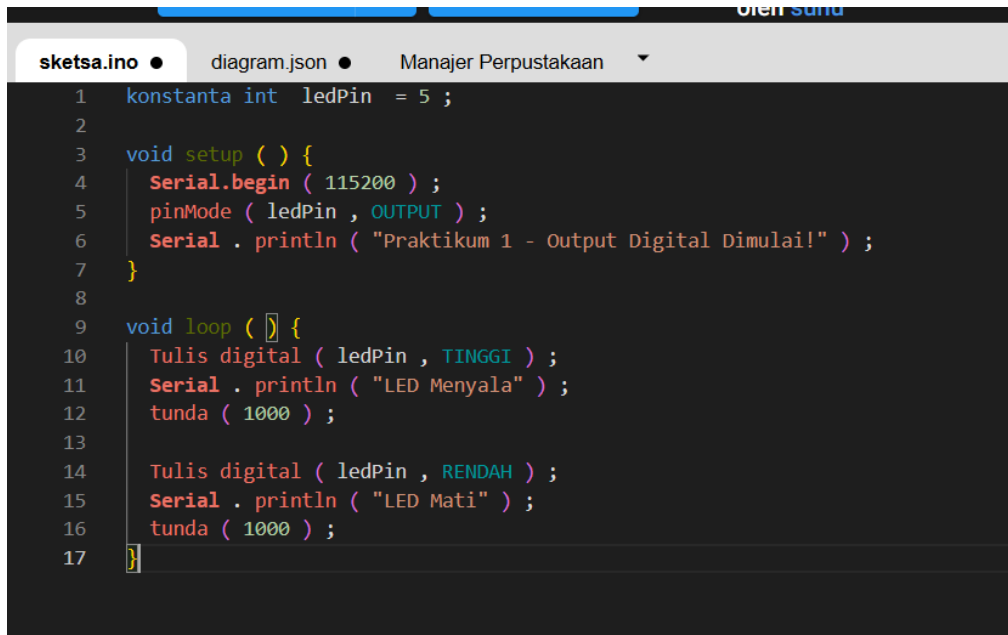


Siklus kerja tersebut dimulai dengan mengalirkan tegangan tinggi ke pin lima sehingga LED menyala, mencetak teks konfirmasi ke layar, lalu menahannya selama satu detik. Selanjutnya, sistem memutus tegangan ke level rendah sehingga LED padam, mengirim teks pemberitahuan bahwa lampu mati, dan kembali menunggu selama satu detik sebelum mengulangi proses yang sama dari awal. Di sisi rangkaian, arus dari pin lima melewati anoda LED, masuk ke resistor pembatas arus untuk mencegah kerusakan komponen, lalu kembali ke pin arde atau *ground*.

Kode diagram.json nya:

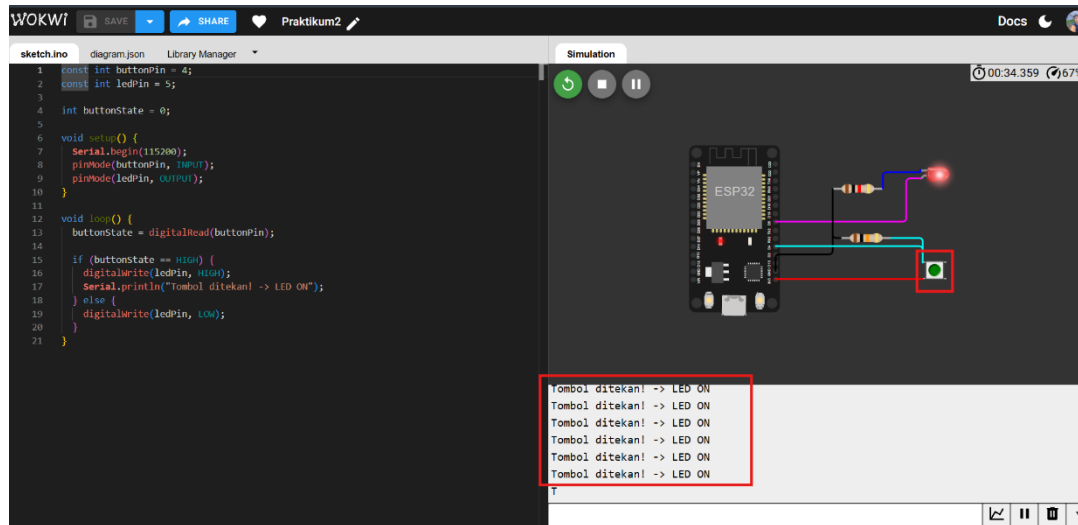


Kode Skecth.ino:



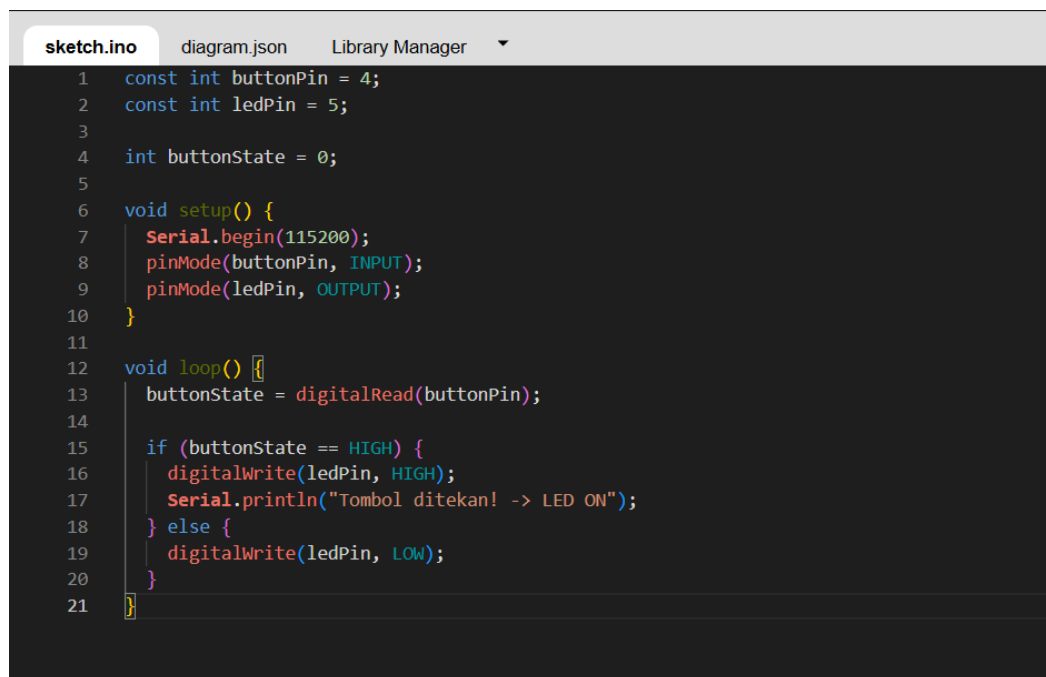
```
1  konstanta int  ledPin  = 5 ;
2
3  void setup ( ) {
4      Serial.begin ( 115200 ) ;
5      pinMode ( ledPin , OUTPUT ) ;
6      Serial . println ( "Praktikum 1 - Output Digital Dimulai!" ) ;
7  }
8
9  void loop ( ) {
10     Tulis digital ( ledPin , TINGGI ) ;
11     Serial . println ( "LED Menyala" ) ;
12     tunda ( 1000 ) ;
13
14     Tulis digital ( ledPin , RENDAH ) ;
15     Serial . println ( "LED Mati" ) ;
16     tunda ( 1000 ) ;
17 }
```

1.5.2 Praktikum 2: *Digital Input (Membaca Push Button)*



Ketika push button ditekan, Maka lampu LED akan otomatis menyala.

Kode Sketch.ino:



Kode diagram.json:

```
1 {
2   "version": 1,
3   "author": "Anonymous maker",
4   "editor": "wokwi",
5   "parts": [
6     { "type": "wokwi-esp32-devkit-v1", "id": "esp", "top": 4.7, "left": -139.4, "attrs": {} },
7     {
8       "type": "wokwi-led",
9       "id": "led1",
10      "top": 15.6,
11      "left": 128.6,
12      "rotate": 90,
13      "attrs": { "color": "red" }
14    },
15    {
16      "type": "wokwi-resistor",
17      "id": "r1",
18      "top": 51.95,
19      "left": 28.8,
20      "attrs": { "value": "1000" }
21    },
22    {
23      "type": "wokwi-resistor",
24      "id": "r2",
25      "top": 109.55,
26      "left": 38.4,
27      "attrs": { "value": "10000" }
28    },
29    {
30      "type": "wokwi-pushbutton-6mm",
31      "id": "btn1",
32      "top": 141.8,
33      "left": 134.4,
34      "attrs": { "color": "green", "xray": "1" }
35    }
36  ],
37  "connections": [
38    [ "esp:TX0", "$serialMonitor:RX", "", [ ] ],
39    [ "esp:RX0", "$serialMonitor:TX", "", [ ] ],
40    [ "esp:D5", "led1:A", "magenta", [ "h153.3", "v-50.4" ] ],
41    [ "led1:C", "r1:2", "blue", [ "v2.8", "h-44" ] ],
42    [ "r1:1", "esp:GND.1", "black", [ "v76.8", "h-66.9" ] ],
43    [ "esp:3V3", "btn1:2.1", "red", [ "v0", "h172.5" ] ],
44    [ "r2:2", "btn1:1.1", "cyan", [ "v0", "h37.2" ] ],
45    [ "btn1:1.1", "esp:D4", "cyan", [ "v-19.2", "h-134.4", "v-0.1" ] ],
46    [ "r1:1", "r2:1", "black", [ "v0" ] ]
47  ],
48  "dependencies": {}
49 }
```

1.6 Latihan

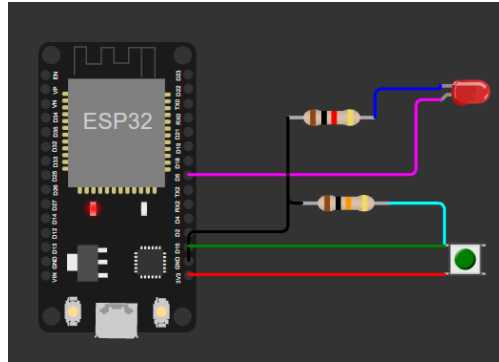
Kerjakan latihan berikut untuk menguji pemahaman Anda. Jelaskan hasilnya di dalam laporan!

- Latihan 1 (Pemahaman Logika Pin): Berdasarkan tabel Referensi Pinout di subbab 1.3.3, jelaskan apa yang mungkin terjadi jika Anda menggunakan logika rangkaian dari Praktikum 2 (Active-High / *pull-down*), namun menyambungkan tombol tersebut ke pin D8 (GPIO 15), lalu menekan tombol tersebut *tepat* pada detik di mana NodeMCU baru diberi daya?

Jawab: NodeMCU akan gagal melakukan *booting* (gagal menyala/menjalankan program) atau masuk ke mode *flash* yang tidak semestinya. Karena Pin D8 (GPIO 15) merupakan salah satu **strapping pin** penting. Agar mikrokontroler dapat melakukan proses secara normal dan menjalankan program dari memori internal, pin GPIO 15 wajib berada boot dalam kondisi logika LOW (terhubung ke GND) saat mikrokontroler pertama kali diberi daya atau di-reset. Pada rangkaian Active-High, pin GPIO dihubungkan ke tombol yang menyambung ke jalur **3.3V (VCC)**. Saat tombol tidak ditekan, resistor pull-down menahan pin di level LOW. Namun, saat tombol ditekan, jalur 3.3V langsung terhubung ke pin sehingga logikanya menjadi **HIGH**.

Jika tombol ditekan tepat pada detik perangkat baru dinyalakan, pin D8 (GPIO 15) akan langsung terbaca sebagai **HIGH**. Kondisi logika HIGH pada GPIO 15 saat fase *startup* melanggar syarat *booting* normal. Akibatnya:

- Proses *booting* normal gagal.
- NodeMCU akan masuk ke mode *SDIO boot* (mode membaca eksternal yang tidak valid), sistem akan macet (*hang*), dan kode program yang ada di dalamnya tidak akan dieksekusi sama sekali.



Gambar Percobaan Latihan 1

- Latihan 2 (Implementasi Kode): Modifikasi kode pada Praktikum 1 agar LED berkedip secara *tidak simetris*, yaitu menyala sangat cepat selama 200 ms, kemudian mati agak lama selama 800 ms. Tuliskan dua baris fungsi `delay()` yang harus Anda ubah.

Jawab:

Baris 12 (durasi menyala):

- Ubah dari `delay(1000);` menjadi = `delay(200)`

Baris 16 (durasi mati):

- Ubah dari `delay(1000);` menjadi = `delay(800)`

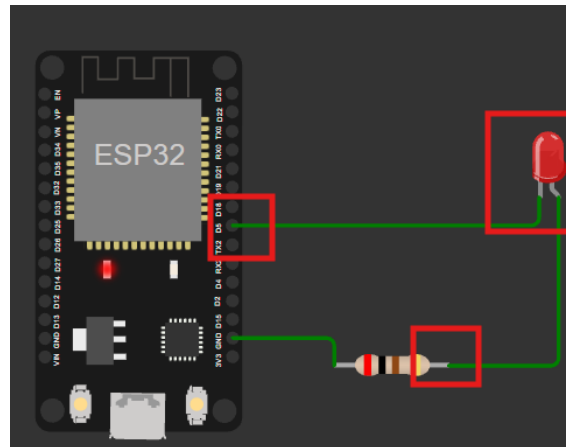
- Latihan 3 (Analisis Sirkuit): Berdasarkan hukum aliran arus listrik searah, apa yang terjadi jika pada Praktikum 1 Anda memasang kaki LED secara terbalik (Anoda ke GND, Katoda ke D1)? Apakah program akan mengalami *error* kompilasi? Mengapa LED tidak menyala?

Jawab: Tidak. Program tetap berhasil dikompilasi (*compile*) dan diunggah tanpa kendala, serta fungsi pengiriman teks ke Serial Monitor tetap berjalan normal. Kompiler (*compiler*) hanya bertugas memeriksa kesalahan sintaksis atau logika pemrograman perangkat lunak, bukan mendeteksi kekeliruan orientasi fisik komponen pada rangkaian elektronika.

Mengapa LED tidak menyala?

LED merupakan komponen semikonduktor berpolaritas yang bekerja sebagai dioda searah. LED hanya akan memancarkan cahaya jika dialiri arus listrik dalam kondisi **panjar maju** (*forward bias*), yaitu saat kaki **Anoda** mendapatkan tegangan yang lebih positif daripada kaki **Katoda**. Ketika dipasang terbalik (Anoda ke GND dan Katoda ke D1 saat berlogika HIGH), komponen berada dalam kondisi **panjar mundur** (reverse

bias). Kondisi ini menciptakan lapisan penipisan (depletion region) yang memiliki hambatan sangat tinggi, sehingga arus listrik searah tersumbat dan tidak dapat mengalir melintasi LED. Akibat ketiadaan aliran arus tersebut, LED tidak akan menyala sama sekali.



Gambar Latihan 3

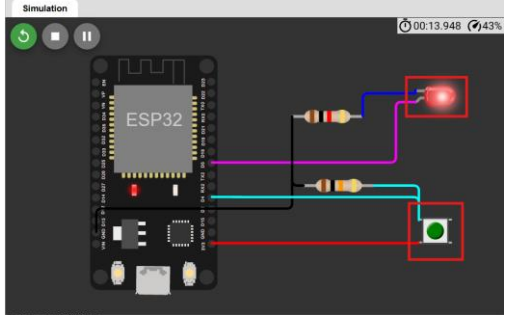
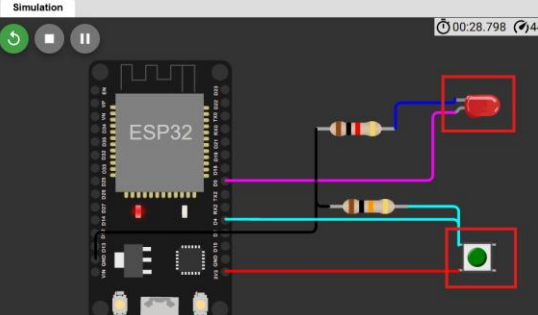
- Latihan 4 (Pengembangan Algoritma): Ubah blok kondisi if-else pada Praktikum 2 sehingga perilakunya terbalik: "Saat tombol TIDAK ditekan, LED menyala. Saat tombol DITEKAN, LED justru mati."

Jawab:

- Baris di dalam if (buttonState == HIGH): dari `digitalWrite(ledPin, HIGH);` diubah menjadi `digitalWrite(ledPin, LOW);`;
- Baris di dalam else: dari `digitalWrite(ledPin, LOW);` diubah menjadi `digitalWrite(ledPin, HIGH);`;

1.7 Tugas

- Instruksi:
 - i. Gunakan konfigurasi *hardware* yang persis sama dengan Praktikum 2 (Rangkaian *Pull-Down*).
 - ii. Modifikasi kode program sehingga tombol berfungsi layaknya sakelar lampu ruangan:
 - Saat sistem pertama dinyalakan, LED dalam keadaan mati.
 - Jika tombol ditekan 1 kali *lalu dilepas*, LED menyala dan tetap menyala.
 - Jika tombol ditekan 1 kali lagi *lalu dilepas*, LED mati dan tetap mati.
 - iii. *Petunjuk*: Gunakan variabel penyimpanan status (misalnya tipe boolean atau int) untuk melacak apakah lampu saat ini sedang ON atau OFF. Anda juga membutuhkan instruksi `delay()` singkat (sekitar 200 milidetik) tepat setelah mendeteksi tombol ditekan, untuk mengatasi fenomena pantulan mekanis (*debouncing* pada tombol fisik).
- Kriteria Keberhasilan:
 - Perubahan status (ON ke OFF, atau sebaliknya) hanya terjadi dengan *satu kali tekanan singkat* (bukan ditahan).
 - Tidak ada fenomena *flickering* (lampu berkedip acak atau gagal mati) saat tombol ditekan.
 - Program memiliki baris komentar yang menjelaskan bagaimana logika pelacakan *state* dan *debounce* bekerja.

 <pre> entry 0x40805dc Sistem ESP32 Siap! Status Awal: LED MATI Tombol ditekan 1x -> LED MENYALA (Terkunci ON) Tombol ditekan 1x -> LED MATI (Terkunci OFF) Tombol ditekan 1x -> LED MENYALA (Terkunci ON) Tombol ditekan 1x -> LED MATI (Terkunci OFF) Tombol ditekan 1x -> LED MENYALA (Terkunci ON) </pre>	 <pre> Sistem ESP32 Siap! Status Awal: LED MATI Tombol ditekan 1x -> LED MENYALA (Terkunci ON) Tombol ditekan 1x -> LED MATI (Terkunci OFF) Tombol ditekan 1x -> LED MENYALA (Terkunci ON) Tombol ditekan 1x -> LED MATI (Terkunci OFF) Tombol ditekan 1x -> LED MENYALA (Terkunci ON) Tombol ditekan 1x -> LED MATI (Terkunci OFF) Tombol ditekan 1x -> LED MENYALA (Terkunci ON) Tombol ditekan 1x -> LED MATI (Terkunci OFF) </pre>
Ketika push button di pencet(on), Lampu akan otomatis menyala terus	Ketika push button di pencet(off), Lampu akan mati

Pembahasan Logika Program pada Laporan

1. Implementasi *State Tracking* (Pelacakan Status):

Penggunaan variabel boolean `ledState` berfungsi layaknya sakelar bistabil (sakelar ruangan), di mana mikrokontroler mengingat (*retain*) kondisi terakhir rangkaian keluaran. Hal ini mengubah sifat alami *push button* yang awalnya bekerja secara sesaat (*momentary switch*) menjadi sistem sakelar pengunci (*latching switch*).

2. Pencegahan Kedip (*Debouncing*):

Instruksi `delay (200);` yang disisipkan setelah proses deteksi penekanan tombol berfungsi meredam fenomena pantulan mekanis (*contact bounce*) pada plat sakelar fisik. Hal ini memastikan siklus pembacaan tidak membaca lonjakan tegangan transien yang dapat mengakibatkan lampu berkedip secara acak (*flickering*) atau gagal berpindah status (*toggle*).